ID730001: Advanced Application Development Concepts

Project

Level	Credits	Assessment Type	Weighting
7	15	Individual	80%

Assessment Overview

In this individual assessment, you will complete two parts:

- **Part 1:** A fully playable command-line Blackjack application that demonstrates object-oriented programming, design patterns, concurrency, and persistent data storage using a relational database.
- **Part 2:** An independently researched tool or framework implemented as a working prototype.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Critically evaluate and implement a range of programming paradigms to solve unfamiliar problems.
2. Create efficient full-stack applications using advanced industry techniques, tools and frameworks.

Assessments

Assessment	Weighting	Due Date	Learning Outcome
Practical	20%	26 June at 4:59 PM	1, 2
Project	80%	26 June at 4:59 PM	1, 2

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment must be completed by 26 June at 4:59 PM.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of 50% across all assessments in ID730001: Advanced Application Development Concepts.

Submission

You must submit all application files via GitHub Classroom.

- Repository URL: https://classroom.github.com/a/RgDe5bs0
- Branch: Switch to the Project branch using the following command: `git checkout -b project`
- Due Date: 26 June at 4:59 PM
- Late Penalty: 10% per day, rolling over at 12:00 AM

The latest application files in the Project branch will be used to mark against the marking rubric. Please test your application before you submit. Partial marks may be given for incomplete functionality.

Authenticity

All parts of your submitted assessment must be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcomes for this assessment.

AI Tools

Learning to use AI tools is an important skill. While AI tools are powerful, you must be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response.
- Do not trust the AI tool's responses blindly. You must still use your judgement and may need to do additional research to determine if the response is correct.
- Acknowledge what AI tool you have used. In the assessment's repository `README.md` file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work.

This also applies to code snippets retrieved from StackOverflow and GitHub. Failure to do this may result in a mark of zero for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with Otago Polytechnic policies. Learners can view policies on the Otago Polytechnic website at https://www.op.ac.nz/about-us/governance-and-management/policies.

Extensions

Familiarise yourself with the assessment due date. Extensions will only be granted if you are unable to complete the assessment by the due date because of unforeseen circumstances outside your control. The length of the extension granted will depend on the circumstances and must be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments are not applicable in ID730001: Advanced Application Development Concepts.

Part 1: Blackjack Application — Development Phase

Learning Outcomes 1 and 2 (28 marks)

Once the course lecturer has approved your design document (submitted as part of the Practical assessment), you can start developing your application. You should follow the requirements in your design document, but you can make changes as needed. If you make significant changes to your design document, please update it accordingly.

Project Management (2 marks)

In this phase, you will use the Agile software development methodology.

- You will decide the duration of your sprints and how to break down your work into sprints and tasks.
- Create a GitHub Project to manage your work. The project must include columns for Backlog, In Progress and Done.
- Create issues for each task and move them across the columns as you work on them.

Here is an example of how to break down your work into sprints and tasks:

Sprint	Tasks
1	Set up project structure, implement `Card`, `Deck`, and `Hand` classes, implement basic `Player` and `Dealer` classes
2	Implement `BlackjackGame` class and core game loop, implement split and surrender actions, implement design pattern 1
3	...
4	...
5	...

Core Game (8 marks)

Implement a fully playable command-line Blackjack game using object-oriented programming. The game must include the following classes:

Class	Required Attributes	Required Methods
`Card`	`rank`, `suit`	`getValue()` - returns the card's numeric value; `toString()` - returns a human-readable representation such as `"Ace of Spades"`
`Deck`	`cards`	`shuffle()`, `dealCard()`, `cardsRemaining()`, `reset()`
`Hand`	`cards`	`addCard(card)`, `getValue()` - correctly handles Ace as 11 or 1; `isBlackjack()`, `isBust()`, `isSoft()` - returns true if the hand contains an Ace counted as 11; `toString()`
`Player`	`name`, `hand`, `score`	`receiveResult(outcome)` - updates score based on outcome; `getScore()` - returns current score; `toString()`
`Dealer`	`hand`, `name`	`showPartialHand()` - reveals only the first card; `showFullHand()`, `shouldHit()` - returns true if hand value ≤ 16; `toString()`
`BlackjackGame`	`deck`, `dealer`, `players`, `currentRound`	`startRound()`, `dealInitialCards()`, `playerTurn(player)`, `dealerTurn()`, `determineWinners()`, `displayGameState()`, `playGame()`, `gameOver()`

> **Note:** Method and attribute names should follow the naming conventions of your chosen language. The names above are illustrative. What matters is that the behaviour matches the descriptions given.

Scoring System

Players accumulate points across rounds based on their outcomes. There are no chips or bets.

Outcome	Points
Blackjack (Ace + 10-value card on initial deal)	+2
Win	+1
Tie / Push	0
Loss	-1
Surrender	-1

- Players start with a score of 0.
- A player's score can go negative.

- A player is eliminated when their score reaches -10.
- The game ends when all players have been eliminated or all players choose to quit.
- A final leaderboard is displayed when the game ends.

Player Actions

In addition to hit and stand, players may take the following actions on their turn:

- **Split:** available when the player's first two cards are a pair. Splits the hand into two separate hands, each receiving an additional card. Each split hand is then played independently.
- **Surrender:** available on the player's first action of a round (before any hits). The player forfeits the round immediately and receives -1 point.

Game Rules

- Goal: get closer to 21 than the dealer without going over, i.e., busting.
- Face cards (J, Q, K) = 10; Aces = 11 or 1; number cards = face value.
- Dealer must hit on ≤ 16 and stand on ≥ 17.
- Players may hit, stand, split, or surrender on their turn.
- A Blackjack (Ace + 10-value card on the initial deal) beats any non-Blackjack 21.
- The game ends when all players have been eliminated or all players choose to quit.

Game Flow

1. Display a welcome message with player names and starting scores.

2. Each round:

   a. Deal two cards to each player and the dealer. The dealer's second card is hidden.

   b. Check for Blackjack.

   c. Each player takes their turn; skip if they have Blackjack.

   d. Dealer reveals hidden card and plays their turn.

   e. Determine and announce winners; update scores.

   f. Remove eliminated players (score ≤ -10).

   g. Ask remaining players whether to play another round.

3. Display a final leaderboard when the game ends.

Example game session:

```
Welcome to Blackjack!
Players: Alice (Score: 0), Bob (Score: 0)

----- Round 1 -----
```

```
Alice's hand: 10 of Hearts, Ace of Spades (Total: 21) - Blackjack!
Bob's hand: 7 of Diamonds, 8 of Clubs (Total: 15)
Dealer's hand: 9 of Hearts, [Hidden]

Bob, what do you want to do? (hit / stand / surrender): hit
Bob's hand: 7 of Diamonds, 8 of Clubs, 5 of Spades (Total: 20)
Bob, what do you want to do? (hit / stand): stand

Dealer reveals: 9 of Hearts, 6 of Diamonds (Total: 15)
Dealer hits and receives 10 of Clubs (Total: 25) - Dealer busts!

Alice wins! (Blackjack) - Score: +2 (Total: 2)
Bob wins! - Score: +1 (Total: 1)

Do you want to play another round? (Alice: yes / Bob: yes)
```

To start the game, your entry point should construct a `BlackjackGame` with a list of player names and begin play:

```
game = new BlackjackGame(["Alice", "Bob"])
game.playGame()
```

Design Patterns (12 marks)

Implement **four** design patterns within your Blackjack application. The patterns must be meaningfully integrated — they should add clear value to the design, not be bolted on artificially. Your implementation must match the patterns described in your approved design document from the Practical assessment.

Compulsory Pattern — Strategy (3 marks)

The Strategy Pattern is compulsory. You must implement it.

Implement a `DrawStrategy` interface with at least **four** concrete strategies that control how a player decides to hit, stand, split, or surrender. Bot strategies must follow the standard Blackjack basic strategy tables provided below.

Strategy	Behaviour
`HumanStrategy`	Prompts the player for input via the command line
`BasicStrategy`	Follows the standard Blackjack basic strategy tables exactly
`ConservativeStrategy`	Stands if the hand value is ≥ 15, never splits, never surrenders; otherwise hits
`AggressiveStrategy`	Stands only if the hand value is ≥ 18, always splits pairs, never surrenders; otherwise hits

Players must be configurable with any strategy at construction time. The game loop must not need to know which strategy a player is using.

Chosen Pattern — Pick One (3 marks)

Select and implement **one** of the following design patterns. Your choice must match what was approved in the Practical assessment.

Option A — Observer Pattern

Implement an `IGameObserver` interface and attach at least **two** concrete observers to the game:

Observer	Behaviour
`Scoreboard`	Tracks wins, losses, ties and surrenders per player and prints a summary at the end of each round
`GameLogger`	Logs every significant game event, e.g., card dealt, player action, winner declared, etc., to a timestamped `.log` file

The `BlackjackGame` class must maintain a list of observers and notify all of them at appropriate points during the game, e.g., after each round ends.

Option B — Factory Pattern

Implement a `PlayerFactory` interface with at least **two** concrete factories:

Factory	Creates
`HumanPlayerFactory`	A `HumanPlayer` whose actions are driven by keyboard input
`BotPlayerFactory`	A `BotPlayer` whose actions are determined automatically using the basic strategy table

The `BlackjackGame` class must accept factories rather than constructing players directly, so that human and bot players can be mixed freely without changing the game logic.

Option C — Builder Pattern

Implement a `GameConfigBuilder` that constructs a `GameConfig` object step-by-step before the game starts. The builder must support at least the following options:

Method	Effect
`setNumPlayers(n)`	Sets the number of players (1–6)
`setNumDecks(n)`	Combines `n` standard 52-card decks into one shoe

Method	Effect
`setEliminationThreshold(n)`	Sets the score at which a player is eliminated (default: -10)
`setTimeLimit(seconds)`	Sets the time limit per player turn in seconds (used with the timed turns concurrency option)
`withVerboseOutput(bool)`	Enables or disables detailed play-by-play output
`build()`	Returns the validated `GameConfig` object; raises an appropriate error for invalid configurations

> **Note:** Method names should follow the naming conventions of your chosen language.

The `BlackjackGame` class must accept a `GameConfig` object and respect all configured options during play.

Additional Design Patterns — Implement Two (6 marks)

Implement the **two additional design patterns** you described in your approved Practical design document. These patterns must be meaningfully integrated into your application, not artificially added.

Your chosen additional patterns must be different from the Strategy Pattern and the one pattern you selected above. All four patterns must be meaningfully distinct from one another.

Blackjack Basic Strategy Tables

Bot strategies must follow these tables. Decisions are based on the player's hand and the dealer's visible up card.

Hard Totals

Hard totals are hands that do not contain an Ace, or contain an Ace that is counted as 1.

Player's Hand Total	Dealer's Up Card	Recommended Action
8 or less	Any	Hit
9	3–6	Hit
9	2, 7–10, A	Hit
10	2–9	Hit
10	10, A	Hit
11	2–10	Hit
11	A	Hit
12	4–6	Stand
12	2–3, 7–10, A	Hit

Player's Hand Total	Dealer's Up Card	Recommended Action
13–16	2–6	Stand
13–16	7–10, A	Hit
17 or more	Any	Stand

Soft Totals

Soft totals are hands that contain an Ace that is counted as 11.

Player's Hand Total	Dealer's Up Card	Recommended Action
A, 2 or A, 3	5–6	Hit
A, 2 or A, 3	2–4, 7–10, A	Hit
A, 4 or A, 5	4–6	Hit
A, 4 or A, 5	2–3, 7–10, A	Hit
A, 6	3–6	Hit
A, 6	2, 7–10, A	Hit
A, 7	2, 7–8	Stand
A, 7	3–6	Hit
A, 7	9–10, A	Hit
A, 8 or A, 9	Any	Stand

Pair Splitting

Player's Hand	Dealer's Up Card	Recommended Action
2, 2 or 3, 3	2–7	Split
2, 2 or 3, 3	8–10, A	Hit
4, 4	5–6	Split
4, 4	Any other	Hit
5, 5	Any	Hit
6, 6	2–6	Split
6, 6	7–10, A	Hit
7, 7	2–7	Split
7, 7	8–10, A	Hit
8, 8	Any	Split

Player's Hand	Dealer's Up Card	Recommended Action
9, 9	2–6, 8–9	Split
9, 9	7, 10, A	Stand
A, A	Any	Split

Surrender

Player's Hand Total	Dealer's Up Card	Recommended Action
15	10	Surrender
16	9–10	Surrender
16	A	Surrender

Concurrency (4 marks)

Add a concurrent feature to your Blackjack application using concurrency tools available in your chosen language, e.g., threads, thread pools, locks, async/await, coroutines. Implement the option you described in your approved Practical design document.

Option A — Concurrent Fraud / Cheat Detection

While the game is running, a background `CheatDetector` task monitors player activity for suspicious patterns. The detector must:

- Run concurrently so it does not block the main game loop.
- Use a thread-safe queue or equivalent mechanism to receive game events from the main thread, e.g., a player hitting 5+ times in a single round, or surrendering every round.
- Use a lock, event, or equivalent synchronisation primitive to safely communicate a detection result back to the main thread.
- Print a warning message during normal output if a suspicious pattern is detected, e.g., `[CheatDetector] Warning: Alice has hit 6 times this round`.
- Not cause any race conditions or deadlocks.

Option B — Timed Player Turns

Each human player has a limited time to make their hit/stand/split/surrender decision. The timer must:

- Run a countdown concurrently with waiting for player input.
- Use a synchronisation primitive, e.g., an event, semaphore, or channel to signal between the input handler and the timer.
- Automatically stand the player (default to `stand`) if the time limit expires before input is received.
- Display a live countdown to the terminal while waiting for input, e.g., `Bob, hit or stand? [8s remaining]`.

- Be configurable — the time limit must be set via the `GameConfig`. The default time limit is 30 seconds if not configured.
- Not cause any race conditions or deadlocks.

Option C — Async Statistics Engine

After each round, a statistics engine computes and displays updated player analytics concurrently so computation does not block the game loop. The engine must:

- Use a thread pool or equivalent concurrent execution mechanism to run at least **three** computations in parallel after each round ends:
  - Win/loss/tie/surrender ratio per player.
  - Running score trend per player across all rounds played.
  - A "hot streak" flag: true if the player has won the last three consecutive rounds.
- Collect and await all results before displaying a formatted stats panel before the next round begins.
- Use a lock or equivalent mechanism to protect the shared statistics store that persists across rounds.
- Not cause any race conditions or deadlocks.

Persistent Statistics with a Relational Database (6 marks)

Implement a `GameDatabase` class that records game history persistently to a relational database, e.g., SQLite, PostgreSQL, MySQL. The database must use the following schema:

```
players      (id, name, created_at)
game_sessions(id, started_at, ended_at, num_rounds)
rounds       (id, session_id, round_number, played_at)
round_results(id, round_id, player_id, outcome, score_before, score_after)
```

`GameDatabase` class:

Method	Description
`constructor(connectionString)`	Opens or creates the database, creates the schema if it does not exist, enables foreign key enforcement where applicable
`getOrCreatePlayer(name) -> id`	Returns the existing player ID if the name is found, otherwise inserts a new player and returns the new ID
`startSession() -> id`	Inserts a new game session record with the current timestamp and returns the session ID
`endSession(sessionId, numRounds)`	Updates the session record with the end timestamp and total rounds played

Method	Description
`recordRound(sessionId, roundNumber) -> id`	Inserts a round record and returns its ID
`recordResult(roundId, playerId, outcome, scoreBefore, scoreAfter)`	Inserts a round_results row for one player's outcome in a round
`getPlayerSummary(name)`	Prints a formatted summary for the named player — see expected output below
`getLeaderboard()`	Prints the top five players ranked by total score accumulated (scoreAfter minus scoreBefore summed across all rounds)

> **Note:** Method names should follow the naming conventions of your chosen language.

The `BlackjackGame` class must use `GameDatabase` to record every session, round, and result as the game is played.

Expected output for `getPlayerSummary("Alice")`:

```
Player: Alice
Total rounds played: 42
Wins: 20  Losses: 16  Ties: 4  Surrenders: 2
Win rate: 47.6%
Best round: +2 (Blackjack, Session 3, Round 7)
Current score: 8
```

> **Hint:** Use transactions for all write operations to ensure data integrity. Always enforce foreign key constraints where your database supports it.

Functional Programming (2 marks)

Implement a standalone module or file, e.g., `blackjackAnalysis`, that analyses a completed game session using functional programming techniques. The module must:

- Accept a collection of round result records with at minimum the following fields: player name, outcome, score before, score after.
- Use the functional programming constructs available in your chosen language — equivalents of `map`, `filter`, and `reduce` — to compute the following:
  - Total score gained across all winning rounds for each player.
  - A list of rounds where the outcome was a surrender.
  - The overall score change per player as a key-value structure.
- Return all results in a single data structure from a function named `analyseSession` or the idiomatic equivalent in your chosen language.

No explicit loops or list comprehensions are permitted within these computations. You must use the functional constructs of your language instead.

Code Quality and Best Practices — Learning Outcomes 1 and 2 (12 marks)

Code Organisation (3 marks)

Organise your code into a clear module or package structure. At minimum, separate concerns into distinct files appropriate to your chosen language:

Responsibility	Contents
Card types	`Card`, `Deck`, `Hand`
Player types	`Player`, `Dealer`, and any strategy/factory classes
Game logic	`BlackjackGame`, observer or builder classes
Database access	`GameDatabase`
Functional analysis module	Functional programming analysis module
Entry point	Constructs and starts the game

- Each module must have a single, clearly defined responsibility.
- Functions and methods must be focused. No function should exceed 30 lines.
- Avoid global state; pass dependencies explicitly.

Code Style and Formatting (2 marks)

- All code must pass a linting tool appropriate to your chosen language with no errors and no significant warnings.
- All code must be formatted consistently using a formatter appropriate to your chosen language.
- Follow the naming conventions of your chosen language consistently throughout.
- Use meaningful, descriptive names for all variables, functions, parameters and classes.
- Include type annotations or type hints on all function signatures where your language supports them.

Use of OOP Principles (2 marks)

- **Encapsulation:** use access modifiers or equivalent conventions to restrict access to internal state and expose it only through methods or properties where appropriate.
- **Inheritance:** at minimum, `Dealer` should inherit from a base class or implement a shared interface with `Player`.
- **Polymorphism:** the game loop must be able to call methods such as `takeTurn()` or `chooseAction()` on different player types without branching on type.
- Avoid code duplication — extract shared logic into base classes or utility functions.

Error Handling (2 marks)

- Wrap all database operations in appropriate error handling constructs and handle database errors gracefully.
- Validate all user input from the command line, e.g., ensure hit/stand/split/surrender input is one of the accepted values; reject invalid options with clear feedback.
- Raise or throw appropriate exceptions with descriptive messages for invalid internal states, e.g., dealing from an empty deck.
- Never expose raw stack traces or internal error details to the player during normal gameplay.

Version Control (2 marks)

- Maintain a clean Git history with descriptive commit messages.
- Use conventional commit messages, e.g., `feat:`, `fix:`, `refactor:`, `docs:`, `test:`, etc.
- Commit regularly with small, focused changes rather than large, monolithic commits.

Part 2: Independent Framework Research — Implementation Phase

Learning Outcomes 1 and 2 (25 marks)

Once your Practical proposal is approved, implement all 12 requirements in a working prototype. Commit all implementation files to the `project` branch in a clearly named subdirectory, e.g., `part2/`.

Marks per Requirement

Each of the 12 requirements is worth marks according to the complexity band agreed during the Practical research and approval phase:

Band	Description	Marks each
Standard	Straightforward use of the framework's core features	1
Intermediate	Requires understanding of more advanced features or integration with other components	2
Advanced	Demonstrates critical evaluation and non-trivial application of the framework	3

Your 12 requirements must include **at least three Advanced requirements**, giving a maximum of 25 marks across the implementation phase. The course lecturer will confirm the band for each requirement at approval time.

Author: Grayson Orr Course: ID730001: Advanced Application Development Concepts